JavaScript – pt.1

Syntax, Data Type, Loops, Control Flow, Functions.

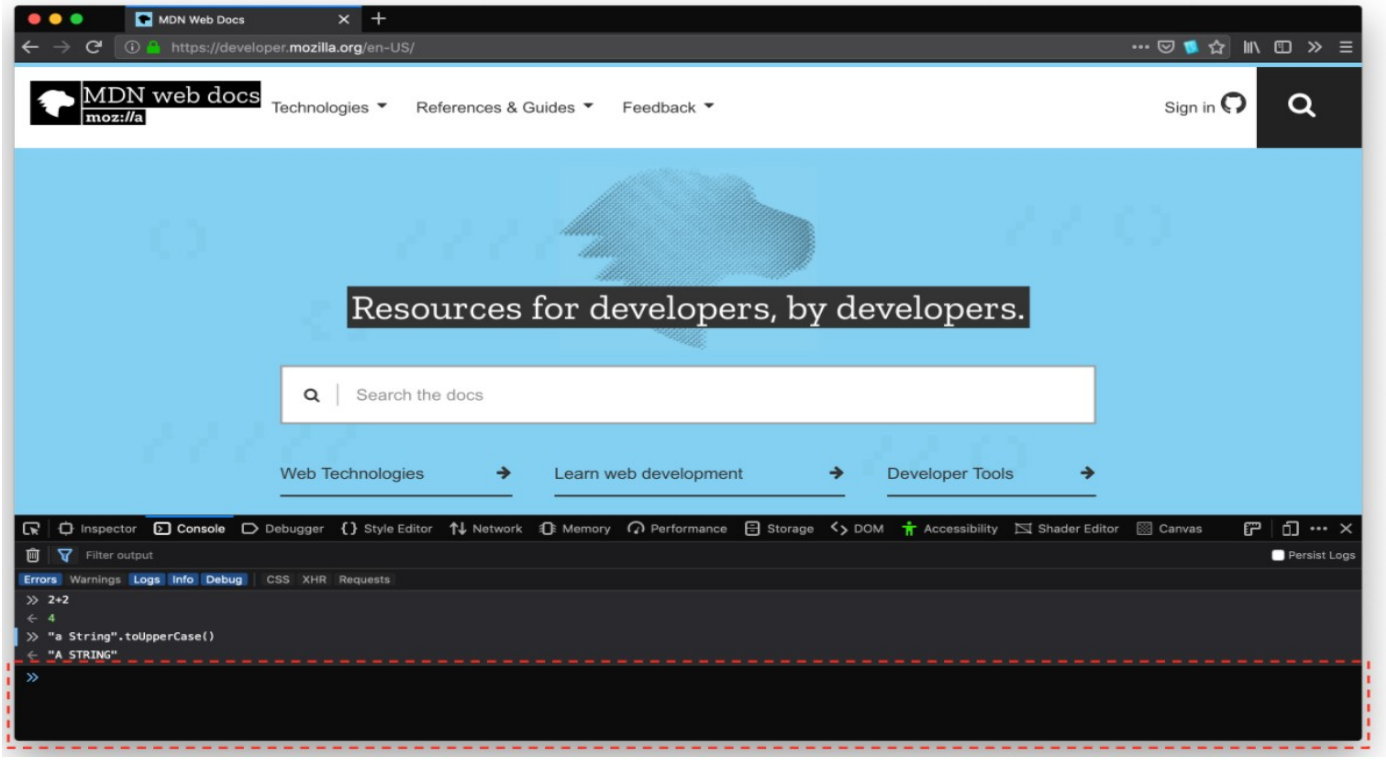


What is JavaScript?

- JavaScript est un langage de script multi-plateforme orienté objet utilisé pour rendre les pages Web interactives (par exemple, avoir des animations complexes, des boutons cliquables, des menus contextuels, etc.)
- Dans un environnement hôte (par exemple, un navigateur Web), JavaScript peut être connecté aux objets de son environnement pour fournir un contrôle programmatique sur eux.
- JavaScript contient une bibliothèque standard d'objets, tels que Array, Date et Math, et un ensemble principal d'éléments de langage tels que des opérateurs, des structures de contrôle et des instructions.
- JavaScript peut être étendu à diverses fins en le complétant avec des objets supplémentaires; par exemple:
  - JavaScript côté client étend le langage de base en fournissant des objets pour contrôler un navigateur et son modèle d'objet de document (Document Object Model).
  - JavaScript côté serveur étend le langage de base en fournissant des objets pertinents pour l'exécution de JavaScript sur un serveur.

Getting started with JavaScript

- Commencer avec JavaScript est simple: il vous suffit d'un navigateur Web moderne.
- La console Web affiche des informations sur la page Web actuellement chargée et comprend également un interpréteur JavaScript que vous pouvez utiliser pour exécuter des expressions JavaScript dans la page en cours.



Basics

- JavaScript emprunte la majeure partie de sa syntaxe à Java, C et C ++, mais il a également été influencé par Perl et Python.
- JavaScript est sensible à la casse et utilise le Unicode character set.
  - Par exemple, le mot ***Früh*** (qui signifie «tôt» en allemand) pourrait être utilisé comme nom de variable.

```
1 | let Früh = "foobar"
```

- La variable ***früh*** n'est pas la même que ***Früh*** car JavaScript est sensible à la casse.
- En JavaScript, les instructions sont appelées « statements » et sont séparées par des points-virgules (;).

Comments

- La syntaxe des commentaires est la même qu'en C ++ et dans de nombreux autres langages:

```
1  // a one line comment
2
3  /* this is a longer,
4   * multi-line comment
5   */
6
7  /* You can't, however, /* nest comments */ SyntaxError */
```

- Les commentaires se comportent comme des espaces et sont ignorés lors de l'exécution du script.

Declarations

JavaScript a trois types de déclarations de variables.

- var : Déclare une variable, en l'initialisant éventuellement à une valeur.
- let : Déclare une block-scoped, locale variable, en l'initialisant optionnellement à une valeur.
- const : Déclare une block-scoped, read-only constante.

Variables

- Vous utilisez des variables comme noms symboliques pour les valeurs dans votre application. Les noms des variables, appelés identificateurs, sont conformes à certaines règles.
- Un identifiant JavaScript doit commencer par une lettre, un trait de soulignement (_) ou un signe dollar ($).
- JavaScript est sensible à la casse, les lettres incluent les caractères «A» à «Z» (majuscules) ainsi que «a» à «z» (minuscules).
- Quelques exemples de noms légaux sont : *Number_hits, temp99, $credit, _name*.

Declaring variables

Vous pouvez déclarer une variable de deux manières:

- Avec le mot-clé ***var***. Par exemple, **var** x = 42. Cette syntaxe peut être utilisée pour déclarer des variables locales et globales, en fonction du *contexte d'exécution*.
- Avec le mot-clé ***const*** ou ***let***. Par exemple, **let** y = 13. Cette syntaxe peut être utilisée pour déclarer une variable block-scope locale.
- Vous pouvez également simplement attribuer une valeur à une variable.
  - Par exemple, x = 42. Cette syntaxe crée une variable globale non déclarée.
  - Il génère également un avertissement JavaScript strict.
  - Les variables globales non déclarées peuvent souvent conduire à un comportement inattendu. Il est donc déconseillé d'utiliser des variables globales non déclarées.

Evaluating variables

- Une variable déclarée à l'aide de l'instruction **var** ou **let** sans valeur assignée spécifiée a la valeur ***undefined***.
- Une tentative d'accès à une variable non déclarée entraîne la levée d'une exception ***ReferenceError***:

```
1  var a;
2  console.log('The value of a is ' + a); // The value of a is undefined
3
4  console.log('The value of b is ' + b); // The value of b is undefined
5  var b;
6  // This one may puzzle you until you read 'Variable hoisting' below
7
8  console.log('The value of c is ' + c); // Uncaught ReferenceError: c is not defined
9
10 let x;
11 console.log('The value of x is ' + x); // The value of x is undefined
12
13 console.log('The value of y is ' + y); // Uncaught ReferenceError: y is not defined
14 let y;
```

Evaluating variables

- Vous pouvez utiliser ***undefined*** pour déterminer si une variable a une valeur.
- Dans le code suivant, l'entrée de variable ne reçoit pas de valeur et l'instruction ***if*** est évaluée à **true**.

```
1  var input;
2  if (input === undefined) {
3    doThis();
4  } else {
5    doThat();
6  }
```

Evaluating variables

- La valeur ***undefined*** est convertie en ***NaN*** lorsqu'elle est utilisée dans un contexte numérique.

```
1  var a;
2  a + 2;  // Evaluates to NaN
```

- Lorsque vous évaluez une variable NULL, la valeur NULL se comporte comme 0 dans les contextes numériques et comme false dans les contextes booléens. Par exemple:

```
1  var n = null;
2  console.log(n * 32); // Will log 0 to the console
```

Variable scope

- Lorsque vous déclarez une variable en dehors de toute fonction, elle est appelée une variable globale, car elle est disponible pour tout autre code du document actuel.
- Lorsque vous déclarez une variable dans une fonction, elle est appelée une variable locale, car elle n'est disponible que dans cette fonction.
- le code suivant consignera 5, car la portée de x est le contexte global. La portée de x n'est pas limitée qu'au bloc d'instruction ***if***.

```
1  if (true) {
2    var x = 5;
3  }
4  console.log(x);  // x is 5
```

- Ce comportement change lors de l'utilisation de la déclaration **let**

```
1  if (true) {
2    let y = 5;
3  }
4  console.log(y);  // ReferenceError: y is not defined
```

Variable hoisting

- Une autre chose inhabituelle à propos des variables en JavaScript est que vous pouvez faire référence à une variable déclarée plus tard, sans obtenir d'exception.
- Ce concept est connu sous le nom de **hoisting**.
- Les variables en JavaScript sont, en un sens, «soulevées» en haut de la fonction ou de l'instruction.
- Cependant, les variables levées renvoient une valeur ***undefined***. Ainsi, même si vous déclarez et initialisez après avoir utilisé ou référencé cette variable, elle renvoie toujours ***undefined***.

```
1  /**
2   * Example 1
3   */
4  console.log(x === undefined); // true
5  var x = 3;
6
7  /**
8   * Example 2
9   */
10 // will return a value of undefined
11 var myvar = 'my value';
12
13 (function() {
14   console.log(myvar); // undefined
15   var myvar = 'local value';
16 })();
```

Global variables

- Les variables globales sont en fait des propriétés de l'objet global.
- Dans les pages Web, l'objet global est *window*, vous pouvez donc définir et accéder aux variables globales à l'aide de la syntaxe *window.variable*.
- Par conséquent, vous pouvez accéder aux variables globales déclarées dans une fenêtre ou un cadre à partir d'une autre fenêtre ou d'un autre cadre en spécifiant le nom de la fenêtre ou du cadre.
- Par exemple, si une variable appelée *phoneNumber* est déclarée dans un document, vous pouvez faire référence à cette variable à partir d'un <iframe> comme *parent.phoneNumber*.

Constants

- Vous pouvez créer une constante en lecture seule avec le mot clé **const**.
- La syntaxe d'un identifiant de constante est la même que celle de tout identifiant de variable: il doit commencer par une lettre, un underscore (_) ou un signe dollar ($) et peut contenir des caractères alphabétiques, numériques ou de soulignement.

```
1 | const PI = 3.14;
```

- Une constante ne peut pas changer de valeur via une affectation ou être déclarée à nouveau pendant l'exécution du script. elle doit être initialisé à une valeur.
- Vous ne pouvez pas déclarer une constante portant le même nom qu'une fonction ou une variable dans la même portée. Par exemple:

```
1  // THIS WILL CAUSE AN ERROR
2  function f() {};
3  const f = 5;
4
5  // THIS WILL CAUSE AN ERROR TOO
6  function f() {
7    const g = 5;
8    var g;
9
10   //statements
11 }
```

Data types

Le langage JavaScript définit huit types de données:

- Sept types de données qui sont des primitives:
  - **Boolean**. vrai et faux.
  - **null**. Un mot-clé spécial indiquant une valeur nulle.
  - **undefined**. Une propriété de niveau supérieur dont la valeur n'est pas définie.
  - **Number**. Un nombre entier ou à virgule flottante. Par exemple: 42 ou 3.14159.
  - **BigInt**. Un entier avec une précision arbitraire. Par exemple: 900719254740992n.
  - **String**. Une séquence de caractères représentant une valeur de texte. Par exemple: "Howdy"
- et **Object**.

Bien que ces types de données soient relativement peu nombreux, ils vous permettent d'exécuter des fonctions utiles avec vos applications.

Les objets et les fonctions sont d’autres éléments fondamentaux du langage. Vous pouvez considérer les objets comme des conteneurs pour les valeurs et les fonctions comme des procédures que votre script peut exécuter.

Data type conversion

- JavaScript est un langage typé dynamiquement. Cela signifie que vous n'avez pas à spécifier le type de données d'une variable lorsque vous la déclarez. Les types de données sont automatiquement convertis selon les besoins lors de l'exécution du script.
- Par exemple, vous pouvez définir une variable comme suit:

```
1 | var answer = 42;
```

- Vous pouvez attribuer à la même variable une valeur de chaîne, par exemple:

```
1 | answer = 'Thanks for all the fish...';
```

- Étant donné que JavaScript est typé dynamiquement, cette affectation ne provoque pas de message d'erreur.

Numbers and the '+' operator

- Dans les expressions impliquant des valeurs numériques et des chaînes avec l'opérateur +, JavaScript convertit les valeurs numériques en chaînes. Par exemple, considérez les déclarations suivantes:

```
1 | x = 'The answer is ' + 42 // "The answer is 42"
2 | y = 42 + ' is the answer' // "42 is the answer"
```

- Avec tous les autres opérateurs, JavaScript ne convertit pas les valeurs numériques en chaînes. Par exemple:

```
1 | '37' - 7 // 30
2 | '37' + 7 // "377"
```

Literals

Les littéraux représentent des valeurs en JavaScript. Ce sont des valeurs fixes, et non des variables, que vous fournissez littéralement dans votre script. Cette section décrit les types de littéraux suivants:

- Array literals
- Boolean literals
- Floating-point literals
- Numeric literals
- Object literals
- RegExp literals
- String literals

Array literals

- Un littéral de tableau est une liste de zéro ou plusieurs expressions, chacune représentant un élément de tableau, entre crochets ([ ]).
- Lorsque vous créez un tableau à l'aide d'un littéral de tableau, il est initialisé avec les valeurs spécifiées comme éléments et sa longueur est définie sur le nombre d'arguments spécifié.
- L'exemple suivant crée le tableau ***coffees*** avec trois éléments et une longueur de trois:

```
1 | let coffees = ['French Roast', 'Colombian', 'Kona'];
```

- Il n'est pas nécessaire de spécifier tous les éléments d'un littéral de tableau.

```
1 | let fish = ['Lion', , 'Angel'];
```

- `fish[0]` is "Lion"
- `fish[1]` is `undefined`
- `fish[2]` is "Angel"

Numeric literals

- Les types Number et BigInt peuvent être écrits en décimal (base 10), hexadécimal (base 16), octal (base 8) et binaire (base 2).

- Quelques exemples de littéraux numériques sont:

```
0, 117, -345, 123456789123456789n            (decimal, base 10)
015, 0001, -0o77, 0o777777777777n            (octal, base 8)
0x1123, 0x00111, -0xF1A7, 0x123456789ABCDEFn  (hexadecimal, "hex" or base 16)
0b11, 0b0011, -0b11, 0b11101001010101010101n  (binary, base 2)
```

Floating-point literals

- Un littéral floating point peut avoir les parties suivantes:
  - Un entier décimal qui peut être signé (précédé de "+" ou "-").
  - Un point décimal (".").
  - Une fraction (un autre nombre décimal).
  - Un exposant.
- La partie exposant est un "e" ou "E" suivi d'un entier, qui peut être signé (précédé de "+" ou "-").
- Un littéral floating point doit avoir au moins un chiffre et soit un point décimal, soit "e" (ou "E"). Par exemple :

```
1  3.1415926
2  -.123456789
3  -3.1E+12
4  .1e-23
```

Object literals

- Un objet littéral est une liste de zéro ou plusieurs paires de noms de propriétés et de valeurs associées d'un objet, entre accolades ({ }).

```
1  var sales = 'Toyota';
2
3  function carTypes(name) {
4    if (name === 'Honda') {
5      return name;
6    } else {
7      return "Sorry, we don't sell " + name + ".";
8    }
9  }
10
11 var car = { myCar: 'Saturn', getCar: carTypes('Honda'), special: sales };
12
13 console.log(car.myCar);   // Saturn
14 console.log(car.getCar);  // Honda
15 console.log(car.special); // Toyota
```

String literals

- Un littéral de string comprend zéro ou plusieurs caractères entre guillemets doubles (") ou simples ('). Une string doit être délimitée par des guillemets du même type.
- Voici des exemples de littéraux de chaîne de caractères:

```
'foo'
"bar"
'1234'
'one line \n another line'
"John's cat"
```

- Vous pouvez appeler l'une des méthodes de l'objet String sur une valeur littérale de chaîne.
- Par exemple, vous pouvez utiliser la propriété *String.length* avec une littéral String:

```
// Will print the number of symbols in the string including whitespace.
console.log("John's cat".length)  // In this case, 10.
```

Using special characters in strings

- En plus des caractères ordinaires, vous pouvez également inclure des caractères spéciaux dans les strings, comme illustré dans l'exemple suivant :

```
1 'one line \n another line'
```

- Le tableau suivant répertorie les caractères spéciaux que vous pouvez utiliser dans les chaînes JavaScript.

Table: JavaScript special characters

Character	Meaning
`\0`	Null Byte
`\b`	Backspace
`\f`	Form feed
`\n`	New line
`\r`	Carriage return
`\t`	Tab
`\v`	Vertical tab
`\'`	Apostrophe or single quote
`\"`	Double quote
`\\`	Backslash character

Block statement

- L'instruction la plus basique est une instruction de bloc, qui est utilisée pour grouper des instructions. Le bloc est délimité par une paire d'accolades:

```
{
  statement_1;
  statement_2;
  :
  statement_n;
}
```

- Les instructions de bloc sont couramment utilisées avec les instructions de flux de contrôle (if, for, while).

```
1  while (x < 10) {
2    x++;
3  }
```

- Ici, {x ++; } est l'instruction de bloc,

Conditional statements

- Une instruction conditionnelle est un ensemble de commandes qui s'exécute si une condition spécifiée est vraie.

- JavaScript prend en charge deux instructions conditionnelles: if ... else et switch.

if...else statement

- Utilisez l'instruction if pour exécuter une instruction si une condition logique est vraie. Utilisez la clause optionnelle else pour exécuter une instruction si la condition est fausse.
- Une instruction if ressemble à ceci:

```
if (condition) {
  statement_1;
} else {
  statement_2;
}
```

- Vous pouvez également composer les instructions en utilisant else if pour que plusieurs conditions soient testées en séquence, comme suit:

```
if (condition_1) {
  statement_1;
} else if (condition_2) {
  statement_2;
} else if (condition_n) {
  statement_n;
} else {
  statement_last;
}
```

Switch statement

- Une instruction switch permet à un programme d'évaluer une expression et de tenter de faire correspondre la valeur de l'expression à une étiquette de cas.
- Si une correspondance est trouvée, le programme exécute l'instruction associée.
- Une instruction switch ressemble à ceci:
- L'instruction break facultative associée à chaque clause case garantit que le programme sort du commutateur une fois que l'instruction correspondante est exécutée, puis continue l'exécution à l'instruction suivant le commutateur.

```
switch (expression) {
  case label_1:
    statements_1
    [break;]
  case label_2:
    statements_2
    [break;]
    …
  default:
    statements_def
    [break;]
}
```

for statement

- Une boucle for se répète jusqu'à ce qu'une condition spécifiée soit évaluée à **false**.
- La boucle JavaScript for est similaire à la boucle for Java et C.

- Une instruction for se présente comme suit :

```
for ([initialExpression]; [conditionExpression]; [incrementExpression])
  statement
```

do...while statement

- L'instruction do ... while se répète jusqu'à ce qu'une condition spécifiée soit évaluée à **false**.
- Une instruction do ... while ressemble à

```
do
  statement
while (condition);
```

- L'instruction est toujours exécutée une fois avant que la condition ne soit vérifiée.

- Si la condition est vraie, l'instruction s'exécute à nouveau. À la fin de chaque exécution, la condition est vérifiée. Lors[illegible]e, l'exécution s'arrête et le contrôle passe[illegible]... while.

```
1  let i = 0;
2  do {
3    i += 1;
4    console.log(i);
5  } while (i < 5);
```

while statement

- Une instruction while exécute ses instructions tant qu'une condition spécifiée est évaluée à **true**.
- Une instruction while ressemble à ceci:

```
while (condition)
  statement
```

- Si la condition devient fausse, l'instruction dans la boucle cesse de s'exécuter et le contrôle passe à l'instruction qui suit la boucle.

```
1  let n = 0;
2  let x = 0;
3  while (n < 3) {
4    n++;
5    x += n;
6  }
```

Exercice

Créer un petit programme en JavaScript où l'utilisateur doit deviner un nombre aléatoire entre 1 et 100. Le programme donnera des indices pour aider l'utilisateur à trouver le bon nombre, comme "trop haut" ou "trop bas".

- Utiliser Math.random() et Math.floor() pour générer un nombre aléatoire entre 1 et 100 et le stocker dans une variable.
- Demander à l'utilisateur de saisir un nombre entre 1 et 100 (vous pouvez utiliser prompt() pour simplifier).
- Fixer un nombre maximal de tentatives, par exemple 10, et indiquer à l'utilisateur combien de tentatives il lui reste.
- Répéter la demande de saisie jusqu'à ce que l'utilisateur trouve le bon nombre. Si le nombre deviné est correct, afficher un message de victoire.

Function declarations

- Une définition de fonction (également appelée déclaration de fonction ou instruction de fonction) se compose du mot-clé *function*, suivi de:
  - Le nom de la fonction.
  - Une liste de paramètres de la fonction, entre parenthèses et séparés par des virgules.
  - Les instructions JavaScript qui définissent la fonction, entre accolades, {...}.

- Par exemple, le code suivant définit une fonction simple nommée ***square:***

```
1  function square(number) {
2    return number * number;
3  }
```

Calling functions

- La définition d'une fonction ne l'exécute pas. Le définir nomme simplement la fonction et spécifie ce qu'il faut faire lorsque la fonction est appelée.

- L'appel de la fonction exécute en fait les actions spécifiées avec les paramètres indiqués. Par exemple, si vous définissez la fonction *factorial( ),* vous pouvez l'appeler comme suit:

```
function factorial(n) {
  if ((n === 0) || (n === 1))
    return 1;
  else
    return (n * factorial(n - 1));
}
```

```
var a, b, c, d, e;
a = factorial(1); // a gets the value 1
b = factorial(2); // b gets the value 2
c = factorial(3); // c gets the value 6
d = factorial(4); // d gets the value 24
e = factorial(5); // e gets the value 120
```

try...catch statement

- L'instruction try ... catch marque un bloc d'instructions à essayer et spécifie une ou plusieurs réponses si une exception est levée. Si une exception est levée, l'instruction try ... catch la rattrape.
- L'instruction try ... catch se compose d'un bloc try, qui contient une ou plusieurs instructions, et d'un bloc catch, contenant des instructions qui spécifient ce qu'il faut faire si une exception est levée dans le bloc try.

```
1  function getMonthName(mo) {
2    mo = mo - 1; // Adjust month number for array index (1 = Jan, 12 = Dec)
3    let months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul',
4                  'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
5    if (months[mo]) {
6      return months[mo];
7    } else {
8      throw 'InvalidMonthNo'; // throw keyword is used here
9    }
10 }
11
12 try { // statements to try
13   monthName = getMonthName(myMonth); // function could throw exception
14 }
15 catch (e) {
16   monthName = 'unknown';
17   logMyErrors(e); // pass exception object to error handler (i.e. your own function)
18 }
```